GoBI, 2025/26, , 1–12
Submitted: 18.03.2026
Report

Sequence analysis

SUCC - Sequence and Umi Clustering-based Correction

Mohit Patil[1] and Nikolaos Mitsopoulos[2]

[1,2]Bioinformatics, LMU & TUM, Munich, Germany
[1]Mohit Patil. mohit.patil@tum.de[2]Nikolaos Mitsopoulos. n.mitsopoulos@tum.de

Abstract

Motivation: PCR amplification in RNA-seq can lead to disproportionate duplication of certain reads, which may result in inflated expression estimates and increases the computational workload for read mapping. Unique molecular identifiers (UMIs) are therefore used to label individual fragments prior to amplification. By using UMIs, it becomes possible to identify reads originating from the same molecule, enabling deduplication and correction of sequencing errors. However, widely used deduplication tools require reads to be mapped to a reference genome before deduplication can be performed. While this approach removes PCR-induced bias in expression estimates, the computational cost of mapping duplicated reads remains. Clustering-based approaches provide an alternative strategy to address this limitation by grouping potential duplicate reads prior to alignment using both UMI information and sequence similarity between reads.
Results: In this report, we present a reliable method to deduplicate reads and correct sequencing errors by clustering both UMI sequences and read anchors. The tool implements a two-cycle clustering strategy, where each cycle is optional and configurable depending on the dataset characteristics. Using simulated data, the method successfully clusters PCR duplicates and constructs consensus sequences for each cluster, enabling the correction of sequencing errors.

Keywords RNA-seq, Sequence Correction, UMI, Clustering, Deduplication

Introduction

RNA sequencing (RNA-seq) is a next-generation sequencing technique used to profile all RNA molecules in a sample, providing a snapshot of the transcriptome [5]. The resulting sequenced reads can, for example, be mapped to a reference genome to quantify gene expression. Since gene expression levels are inferred from read counts, it is crucial that sequencing provides an accurate representation of the transcriptome.

A vital step in RNA-seq is the PCR amplification of fragments [1]. This step may lead to over-amplification of certain reads and introduce bias into the read counts, which may cause to false results in downstream analyses like differential expression. Additionally these duplicated reads have no practical yield and create more workload for the read mapper.

To address this issue, unique molecular identifiers (UMIs) are used to correct for PCR amplification bias and to remove PCR duplicates. UMIs are typically 6–12 nucleotide-long sequences that are attached to fragments prior to PCR amplification in order to uniquely label the original molecules. During sequencing, reads that originate from the same molecule therefore carry the same UMI. After mapping the reads to a reference genome, reads that share both the same mapping position and the same UMI are assumed to originate from the same original molecule. This makes it possible to identify and collapse PCR duplicates and to estimate the duplication rate of fragments, i.e., the number of reads derived from the same original molecule [?].

Thus, most current methods, such as UMI-tools, perform UMI-based deduplication after mapping the reads to a reference genome [4]. This approach allows reads that share both the same genomic position and the same UMI to be identified as PCR duplicates and collapsed into a single representative read. However, mapping large numbers of sequencing reads is computationally expensive in terms of both time and memory, particularly since many PCR-duplicated reads are ultimately discarded during the deduplication step.

An alternative approach is to cluster reads prior to mapping using their UMIs in order to identify reads that originate from the same molecule and collapse them into a single representative read. By reducing the number of redundant reads before alignment, this strategy may decrease the computational resources required for downstream analysis.

However, UMIs are not as reliable as they may appear at first glance. Like the sequencing reads themselves, PCR or sequencing errors can change the UMI sequence, creating artificial new UMIs. In addition, even with 12-nucleotide-long UMIs there are only 4^{12} possible combinations, which limits the number of fragments that can be uniquely labeled. As

a result, UMIs cannot uniquely identify all fragments, and multiple fragments may receive the same UMI. Therefore, simply clustering reads based on their UMIs is not sufficient for correcting PCR amplification bias, since the same UMI can be assigned to multiple distinct fragments that are subsequently PCR-amplified.

To address these limitations, we employ an approach that clusters reads using both their UMIs and a part of the read sequences themselves. By incorporating sequence similarity in addition to the UMI information, this method allows sequencing and PCR-induced errors in UMIs and reads to be corrected while simultaneously identifying and collapsing PCR duplicates.

Program Specifications

The tool is provided as an executable Java `.jar` file and can be run from the command line. The available parameters are:

- `-reads`: Path to the FASTQ file containing the read sequences.
- `-umi`: Path to the FASTQ file containing the UMI sequences.
- `-kmer_size`: Length of positional k-mers used for clustering.
- `-read_length`: Length of the reads in the input FASTQ file.
- `-umi_length`: Length of the UMIs in the input FASTQ file.
- `-threshold`: Maximum allowed error rate in the read sequence.
- `-outDir`: Output directory where all result files are stored, including clustering results, correction statistics, and intermediate outputs from both clustering cycles.

Methods

The workflow consists of four main steps. First, the UMIs are read and grouped based on their unique sequences. Next, the reads are clustered using their last 50 nucleotides as anchors in combination with the UMI clusters. Subsequently, the UMIs are corrected, and finally, an additional finer clustering step is performed to account for errors in the last 50 nucleotides.

Clustering UMIs and Anchors

To correct PCR and sequencing errors in both the UMIs and the reads, the program first groups identical UMIs to obtain a set of unique UMI sequences. Subsequently, reads are clustered using the last 50 nucleotides of each read as an anchor sequence, forming anchor clusters. Within each anchor cluster, reads are further divided into subclusters according to their associated UMI. To assign a UMI to an appropriate subcluster, a quality-aware UMI clustering procedure is applied.

Quality-Aware UMI Clustering

Within each anchor cluster, UMIs are grouped using a quality-aware clustering approach that incorporates the Phred quality scores of the UMI sequence. Phred scores estimate the probability that a base has been sequenced incorrectly.

Using these scores, up to two positions in the UMI are selected where the probability of sequencing error exceeds 1% (Phred score below 20). These positions are considered candidate error sites. The bases at these positions are then systematically mutated, and the resulting mutated UMIs are compared against the existing subclusters within the anchor cluster.

If multiple mutated UMIs match existing subclusters, the subcluster with the largest number of reads is selected as the most likely origin of the UMI. The read is then assigned to this subcluster.

The implementation uses a hashing-based lookup to efficiently identify matching subclusters. For each candidate mutation, the modified UMI sequence is used as a key to query a hash table containing the existing subcluster representatives. If a match is found, the corresponding subcluster is retrieved. After the appropriate subcluster has been determined, the original UMI sequence is inserted into the hash table and associated with the identified subcluster to allow efficient assignment of subsequent reads.

Building the consensus Sequence

After quality-aware UMI clustering, a consensus UMI sequence is constructed for each subcluster. The consensus is determined using a quality-weighted scoring scheme that incorporates the Phred quality scores of the bases.

For each position in the UMI, bases contribute support proportional to their Phred quality score and the weight of the corresponding UMI cluster. If multiple reads support the same nucleotide at a given position, their quality scores are accumulated to increase the support for that base. When a different nucleotide is observed, it replaces the current consensus base only if its score exceeds the accumulated score of the existing base. The nucleotide with the highest overall support is therefore selected as the consensus base at that position.

An analogous approach is applied to construct consensus read sequences.

Finer Clustering

After performing the aforementioned steps, the algorithm optionally executes an additional clustering cycle to account for reads that were incorrectly separated due to sequencing errors in the last 50 nucleotides. Given a sequencing error rate of approximately 1%, the probability of observing at least one error within a 50 nucleotide fragment is about 39%. As a result, many reads originating from the same true fragment may contain errors in this region. This typically leads to one large cluster representing the correct 50 nucleotide sequence and several smaller clusters representing sequences containing single-base errors. To address this, a second clustering cycle is performed.

The procedure follows a greedy strategy in which clusters are represented by anchor seeds. This avoids maintaining a consensus sequence for each cluster or computing pairwise distances between all reads, both of which would be computationally prohibitive. Because of the greedy nature of the algorithm, the resulting clustering depends on the input order of the sequences. To mitigate this effect and improve clustering quality, subclusters are sorted by their read counts prior to the finer clustering cycle.

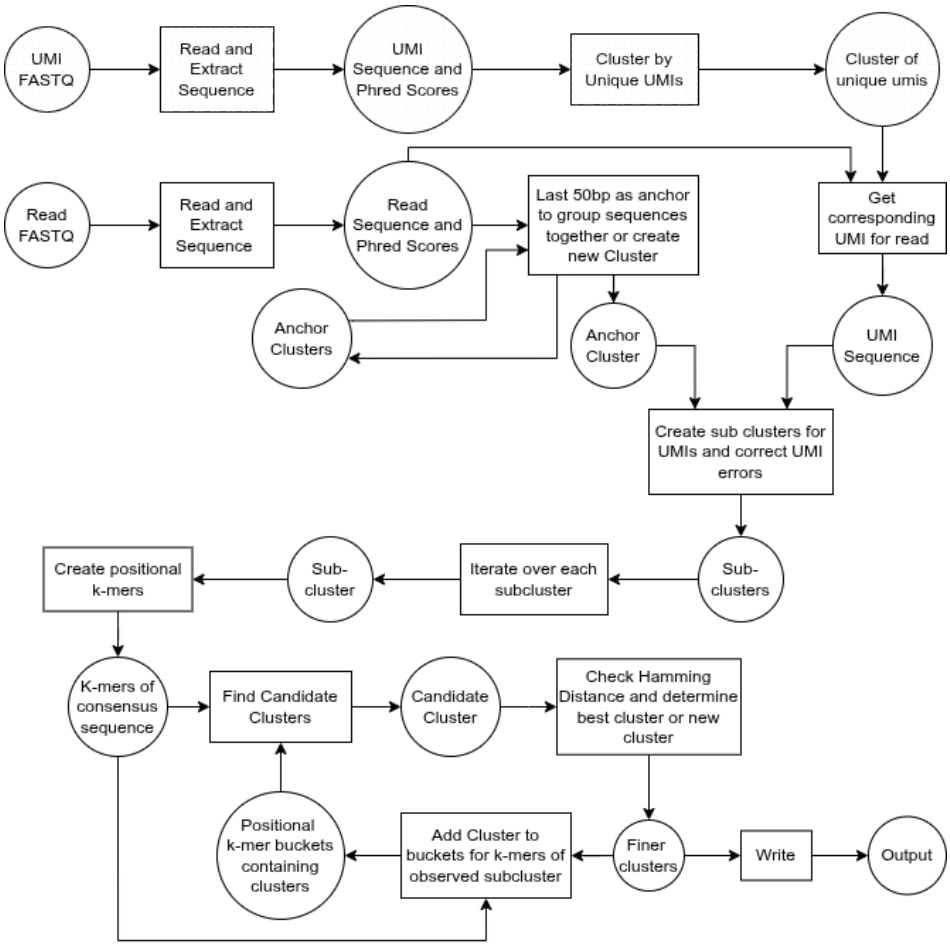

Figure 1 Petri net of the general workflow of the clustering based correction. Circular nodes represent places, and rectangular nodes represent transitions.

Positional k-mer indexing

For the finer clustering cycle, positional k-mer sets are used to efficiently identify candidate clusters. Each positional k-mer set is obtained by dividing the read into equal-length sections and extracting the k-mer of each section. This approach is conceptually similar to the 50 nucleotide clustering strategy described earlier, but extends the idea to the entire read.

The resulting positional k-mer set serves as a key for candidate filtering. Instead of iterating over all clusters to identify potential matches, the algorithm queries only clusters that share a specific k-mer at a given position. For each positional k-mer, the corresponding cluster bucket is retrieved, and the union of these buckets defines the candidate set. This substantially reduces the number of cluster comparisons required.

Admittedly, this method still leaves some room for error. However, the probability of failing to detect the correct cluster decreases rapidly as the k-mer size is reduced. Clustering with smaller k-mers is less likely to be disrupted by errors, since a single nucleotide substitution affects just on k-mer. As a result, even if some k-mers are destroyed by errors, many others remain intact and can still be used to retrieve the correct candidate clusters.

Consequently, the likelihood that all relevant positional k-mers are corrupted simultaneously becomes very small, ensuring that the correct cluster will almost always be included in the candidate set.

Final assignment

After obtaining the cluster candidates, a final filtering step determines whether the query read is assigned to an existing cluster or forms a new one. Several distance functions were evaluated for this step, as it represents a critical component of the clustering process. The measure that produced the most consistent results while remaining computationally inexpensive was the Hamming distance between the query read and the cluster representative.

If the distance exceeds a user-defined threshold, the read forms a new cluster. Otherwise, it is assigned to the cluster whose representative yields the smallest distance.

Clustering modes

The clustering algorithm can operate in different modes depending on whether UMI sequences are available. In the default configuration, clustering is performed in two successive cycles.

In the first cycle, reads are grouped based on their UMI sequences and the last 50 nucleotides of the read, which serve as an anchor region. This step forms initial anchor clusters that group together reads originating from the same fragment while accounting for potential sequencing errors in the UMI sequences.

In the second cycle, these initial clusters are further refined using positional k-mer sets derived from the full read sequence. Candidate clusters are retrieved through positional k-mer matching, and reads are assigned to clusters using the distance-based filtering procedure described previously. This additional clustering step helps merge clusters that were separated in the first cycle due to sequencing errors within the anchor region.

If UMI sequences are not provided, the algorithm operates in a reduced mode in which only the second clustering cycle is executed. In this case, clustering relies solely on positional k-mer matching and sequence similarity, without incorporating UMI information.

Experimental setup

Simulated dataset

The algorithm was evaluated using a simulated RNA-seq dataset to obtain ground-truth cluster assignments. Reads were simulated from the Sus scrofa genome using a modified version of the read simulator implemented for assignment 2.

The simulator generates fragments from annotated transcripts according to predefined transcript counts and produces FASTQ files containing the corresponding forward and reverse reads of a specified length. The simulation framework was extended to model the effects of PCR amplification on the final sequencing results, including exact fragment duplication and stochastic variation in fragment abundance.

When PCR simulation is enabled, the simulator additionally outputs a FASTQ file containing the UMI sequences associated with each read. UMI sequences are sampled uniformly at random from a predefined pool of possible UMIs. The size of this pool, which determines the maximum number of unique UMIs available, can be specified by the user. Because the number of distinct UMI sequences is limited by their length, this sampling strategy can lead to UMI collisions, where multiple fragments receive the same UMI.

This UMI sampling approach implicitly assumes that UMI sequences are drawn without replacement and that duplicates are distributed approximately uniformly across the available UMIs. In practice, real sequencing experiments do not strictly follow this assumption, and the number of duplicate UMIs per unique UMI can vary.

In contrast to the read sequences, UMI sequences are not subject to mutation during the simulation. Furthermore, the simulator does not output the entire set of PCR-amplified fragments. Instead, a subset of reads is sampled according to the specified flow cell capacity parameter, thereby modeling the limited sequencing capacity of the flow cell.

Simulation parameters

To approximate realistic gene expression, the transcript counts were generated using the Sus scrofa RNA sequencing data on the server. Arbitrarily, the minimap2 mapping of sample H1-12936-T2 was chosen. Using the BAM-features analyzer program, transcript counts were extracted from the mapping, removing duplicate counts produced by PCR. Reads that could not be matched to at least one transcript were discarded, as the read simulator can not simulate intronic or merged-transcriptomic read data. When a read was mapped to multiple transcripts, one transcript was selected uniformly at random from the set of candidate transcripts, and the count of the selected transcript was incremented.

Sequencing errors were introduced with a per-base mutation rate of 1%, reflecting typical Illumina sequencing error profiles [2]. In addition, PCR amplification was simulated with an efficiency of 85% over 18 cycles, producing duplicate fragments from the same initial molecule. Each fragment was assigned a 12 bp UMI prior to PCR amplification. The number of unique UMIs was limited to 8500, as this was also the number in the Sus scrofa dataset.

The final dataset contained approximately 20 million reads originating from roughly 6 million unique fragments. Due to the sampling step that models the limited flow cell capacity, only about 4 million unique fragments were ultimately represented in the simulated reads. Consequently, approximately one-third of the original fragments generated during PCR amplification were not included in the final dataset. Table 1 summarizes the parameters of the simulation.

Table 1. Summary of the simulated RNA-seq dataset used for evaluation.

Total reads	19,998,423
Read length	150 bp
UMI length	12 bp
Sequencing error rate	1%
PCR cycles	18
PCR efficiency	85%
Unique fragments	6,174,255
Unique UMIs	8,500

Evaluation metrics

To evaluate the quality of the clustering results, the simulated dataset was used, since the ground truth fragment assignments for each read are known. This allows the predicted clusters to be directly compared to the true fragment origins of the reads.

The true clusters were defined as follows: Two reads belong to the same cluster if and only if they map to exactly the same genomic regions and share the same UMI.

Clustering performance was assessed using several commonly used clustering evaluation metrics, including the Adjusted Rand Index (ARI), pairwise precision, pairwise recall, and the pairwise F1-score. In addition, information-theoretic measures such as homogeneity, completeness, and the V-measure were computed.

Results and Discussion

To test the workflow, the program was applied separately on forward (R1) and reverse reads (R3) of 9 different samples from Sus scrofa. Across all input FASTQ files, the UMIs are 12 nucleotide long and the reads are 150 nucleotides long. The samples have variable sizes:

UMI correction

The quality-aware UMI correction yields the number of unique UMIs shown in Table 2. For all samples, the total number of distinct UMIs decreases slightly after correction compared to the counts observed before correction.

Across the dataset, the reduction in the number of unique UMIs is relatively small, indicating that only a limited fraction of UMIs were modified during the correction step. This pattern is consistent across all samples. A small difference between the corrected UMI counts obtained from R1 and R3 can also be observed for each sample, with R3 consistently showing slightly fewer UMIs than R1 after correction.

Table 2. Approximate number of reads and UMIs across all nine samples, before and after correction.

Sample	No. of Reads	UMIs before correction	UMIs after correction R1	R3
H1	45.23 M	8,551	8,366	8,316
H2	41.20 M	20,023	19,811	19,677
H3	69.03 M	19,322	19,066	18,877
H4	77.66 M	11,880	11,650	11,526
H5	27.61 M	6,991	6,875	6,822
H6	73.38 M	21,081	20,885	20,685
H7	79.67 M	11,411	11,198	11,099
H8	84.07 M	18,009	17,780	17,672
H9	74.43 M	10,325	10,135	10,067

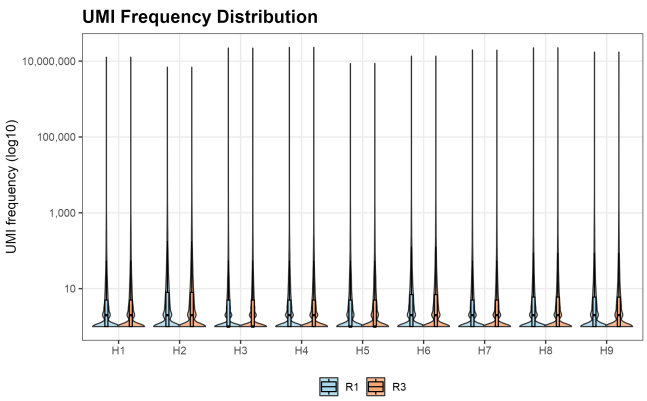

Figure 2 Violin plots showing the distribution of UMI frequencies after correction for each sample. The y-axis is shown on a logarithmic scale. Blue violins correspond to R1, whereas orange violins correspond to R3.

According to figure 2, the majority of corrected UMIs are associated with relatively low read counts across all samples. The distributions are strongly skewed, with most UMIs concentrated at the lower end of the count range. In addition, all samples exhibit a pronounced upper tail extending toward very large counts, reaching up to approximately 10 million reads for a small number of UMIs. This pattern is consistently observed for both R1 and R2 across the dataset.

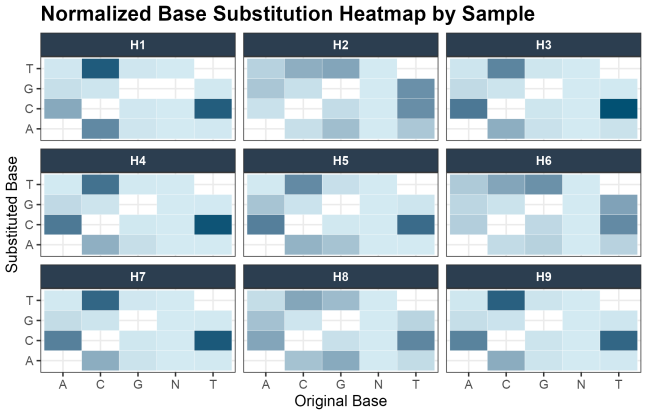

Figure 3 Heatmap showing the types of base substitutions applied during UMI correction for each sample. The x-axis represents the original nucleotide observed in the UMI, while the y-axis represents the nucleotide to which it was corrected. Each cell indicates the normalized frequency of observed substitutions of a given type within each sample, correcting for sample size differences.

The heatmap in figure 3 shows the type of substitutions performed by the program during the UMI correction. The most common corrections are T to C, C to T and A to C. Overall, the majority of corrections correspond to single-nucleotide substitutions between canonical bases, including both transitions and transversions. In contrast, other substitution types occur only rarely.

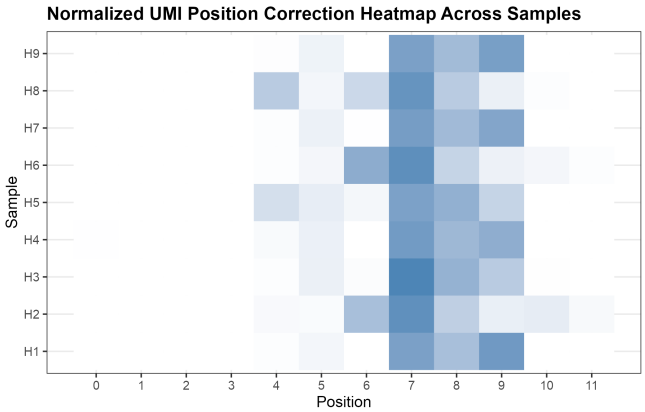

Figure 4 Heatmap showing the frequency of UMI base position corrections across samples. The x-axis represents the base position within the UMI sequence, while the y-axis represents the sample identity. Each cell indicates the normalized frequency of observed corrections at the corresponding position for each sample, accounting for differences in sample size.

According to figure 4, the frequency of UMI base corrections varies across positions within the UMI sequence. The highest correction frequencies are observed at positions 7–9, which show consistently higher normalized correction values across most samples. In contrast, the remaining UMI positions exhibit comparatively lower correction frequencies.

The results demonstrate that quality-aware UMI correction yields only a modest reduction in the number of unique UMIs,

indicating the algorithm preserves molecular diversity while minimizing erroneous merging. This consistent reduction across all samples supports the robustness and reliability of the method. Substitution patterns and position-specific correction hotspots show sensitivity to common error types without excessive modification. Since UMIs are used to distinguish between reads with highly similar sequences, especially those identical in the last 50 bp, they are essential for preventing the merging of reads from paralogous genes or distinct splice variants of the same gene. Careful correction enables accurate identification of unique molecular events, maintaining isoform and gene specificity. The implemented procedure sustains this specificity, preserving the resolution necessary for downstream isoform analysis while ensuring that true biological diversity is maintained.

The consensus-building approach for both UMIs and read sequences is currently based on a relatively simple scoring scheme, which assumes that the majority of supporting bases reflect the true underlying sequence. A potential limitation of this approach arises in cases where distinct fragments share identical anchor sequences (i.e., the last 50 nucleotides) and, after correction, also converge to the same UMI. In such cases, reads originating from different molecules could be erroneously merged.

However, this scenario is expected to occur only rarely in typical RNA-seq data. Transcriptome sequencing generally targets expressed regions with sufficient sequence diversity, and the combined use of anchor sequences and UMIs provides a strong level of discrimination between molecules. While collisions may occur in highly expressed genes or low-complexity regions, the overall likelihood of identical anchors and near-identical UMIs coinciding remains low. Therefore, the impact of such events on downstream analyses is expected to be limited [?].

Clustering Performance

Deduplication

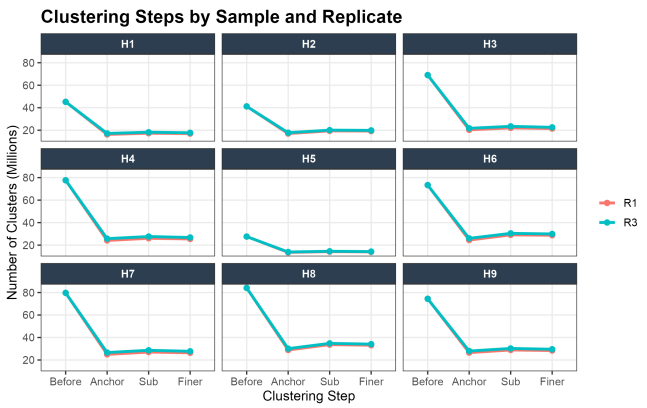

Figure 5 Line plot showing the number of clusters (in millions) at each deduplication and correction stage for all samples and replicates. Each facet corresponds to a sample, while colored lines represent the R1 and R3 replicates. The four clustering stages are: Before (number of reads prior to clustering), Anchor (number of anchor clusters), Sub (number of UMI subclusters), and Finer (final number of clusters after finer clustering).

According to the line plot in figure 5, the total number of reads decreases substantially after the clustering steps for all samples. Across the dataset, the number of reads is reduced by approximately half compared to the counts observed before clustering. This reduction is consistently observed across all samples, indicating that a substantial number of reads are collapsed during the clustering and deduplication process. In addition, a small but consistent difference between the read counts of R1 and R3 can be observed after clustering, with R3 generally retaining slightly more reads than R1. This difference is also reflected in the values reported in table 4 in the supplementary material.

The observed reduction in total read counts following clustering, with approximately half of the initial reads retained and a consistent difference between R1 and R3 across samples, reflects the effectiveness of the deduplication process and inherent technical variations in paired-end sequencing. Specifically, the modestly higher retention of reads in R3 compared to R1 can be attributed to strand-specific differences in sequencing quality, error profiles, and sequence complexity, which are known to influence clustering and deduplication efficiency. These findings are consistent with established reports showing that technical biases arising during library preparation and sequencing can lead to such disparities in read processing outcomes [3].

Furthermore, the number of anchor clusters is consistently larger than the number of UMI subclusters. This indicates that multiple UMI subclusters can be associated with the same anchor sequence, meaning that reads sharing the same anchor (derived from the last 50 nucleotides of the read) may still differ in their associated UMI sequences.

Distribution of Cluster Sizes

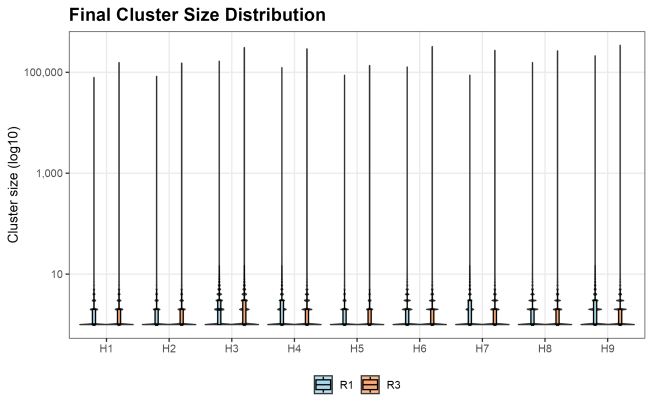

Figure 6 Violin plots showing the distribution of clusters sizes for each sample. The y-axis is shown on a logarithmic scale. Blue violins correspond to R1, whereas orange violins correspond to R3.

According to figure 6, most clusters consist of a small number of reads. The distribution is strongly skewed, with a long upper tail extending to cluster sizes of up to approximately 100,000 reads. This indicates that while the majority of clusters are small, a small number of clusters contain substantially more reads. This pattern is consistently observed across all samples, with clusters in R2 generally reaching slightly larger sizes than those in R1.

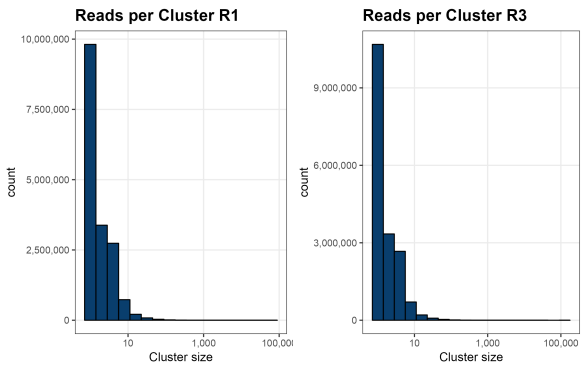

Figure 7 Histograms showing the distribution of cluster sizes for sample H1. The left panel corresponds to R1, while the right panel corresponds to R3. The x-axis in both plots is shown on a logarithmic scale.

A closer inspection of the cluster size distribution, e.g. sample H1 (figure 7) shows that the majority of clusters consist of singletons. Clusters containing more than one read are also observed, with moderate frequencies for cluster sizes up to approximately 10 reads. Beyond this point, the number of clusters decreases sharply, and only a very small number of clusters reach larger sizes.

The Violin plots and histograms reveal that singletons dominate, reflecting the diversity of transcript sequences commonly observed in RNA-seq data. The clear drop-off in cluster frequency as size increases demonstrates that the majority of transcript fragments are either rare or unique, but a small number of clusters, which correspond to highly abundant transcripts or repetitive elements, are much larger. This pattern is consistently observed across all samples analyzed, attesting to the robustness and stability of the clustering algorithm's partitioning regardless of sample-specific variation.

Such heavy-tailed distributions are characteristic in RNA-seq studies and are further amplified by biases introduced during PCR amplification. PCR can preferentially amplify certain fragments, leading to the over-representation of those in sequencing data, and contributing to the long tail of large clusters [3].

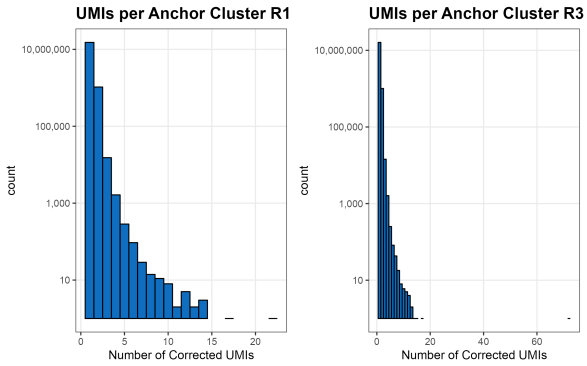

Figure 8 Distributions of anchor cluster sizes and number of subclusters i.e. corrected UMIs per anchor cluster for sample H1. The left panel corresponds to R1, while the right corresponds to R3. The y-axis in both plots is shown on a logarithmic scale.

The distributions shown in Figure 8 indicate that the number of subclusters, i.e. corrected UMIs, per anchor cluster is generally low across the dataset. Most anchor clusters contain only a single or a small number of subclusters. However, a noticeable number of anchor clusters contain more than one subcluster. This observation is consistent with the overall counts reported previously, where the number of anchor clusters exceeds the number of UMI subclusters. Additionally, it can be observed that R3 contains anchor clusters with more than 60 subclusters, whereas in R1 the maximum number of subclusters per anchor cluster is slightly above 20.

The distribution of corrected UMIs per anchor cluster indicates that most clusters encompass only a single or a small number of subclusters, supporting the effectiveness of the clustering method in capturing distinct molecular events. Instances where anchor clusters contain multiple subclusters may represent genuine biological diversity, such as different splice variants or paralogous gene products, or minor sequence variation. The increased number of subclusters per anchor cluster observed in R3 relative to R1 suggests that R3 applies less stringent criteria, thereby grouping reads with greater sequence variation. While this approach can enhance the detection of underlying isoform diversity, it may also increase the risk of conflating molecules from different sources. These findings underscore the importance of selecting appropriate correction stringency to optimize isoform resolution.

Evaluation of Clustering

Simulated reads analysis

Although the simulation was designed to closely resemble the parameters of real RNA-seq experiments, analyzing and validating its output is as important as evaluating the clustering results themselves. Several differences were observed between the simulated data and the real dataset, particularly in the distribution of reads per UMI. These discrepancies may be explained by differences in PCR parameters, simplifying assumptions made during read generation, or potential quality issues in the UMI batch of the H1 dataset.

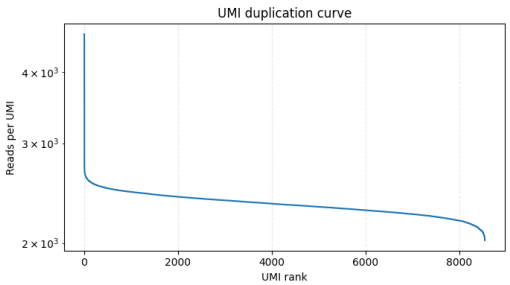

Figure 9 UMI duplication curve of the simulated dataset. UMIs are sorted by their read abundance, and the number of reads per UMI is plotted as a function of UMI rank. This representation highlights the gradual decrease in read counts across UMIs, with highly abundant UMIs appearing at low ranks and less abundant UMIs at higher ranks.

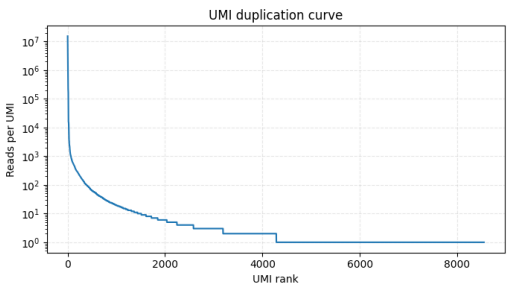

Figure 10 Log-scaled UMI duplication curve of the Sus scrofa dataset. UMIs are ranked by their read counts, and the number of reads per UMI is shown on a logarithmic scale. The distribution is strongly skewed: a small number of UMIs accumulate very high read counts, whereas the majority of UMIs are associated with only a few reads.

A comparison of the UMI duplication curves for the simulated dataset and the H1 dataset reveals notable differences. In the simulated data (figure 9), the number of reads per UMI decreases gradually across UMI ranks, resulting in a relatively smooth duplication curve. In contrast, the duplication curve of the H1 dataset (figure 10) is strongly skewed, with a small number of UMIs accumulating disproportionately large read counts.

One possible explanation for this discrepancy is a difference in PCR amplification conditions. The simulation was performed with 18 PCR cycles, which may not produce pronounced PCR "jackpot" effects observed in real RNA sequencing data, although the effect of PCR cycles on amplification distortions has been questioned ??. Another potential factor is the distribution of UMIs in the sequencing library. If the UMI batch used in the H1 dataset contained unequal abundances of individual UMIs, this could lead to a non-uniform assignment of UMIs to fragments and consequently a more skewed duplication curve.

However, these differences are not expected to significantly influence the clustering itself. The clustering algorithm primarily relies on sequence similarity and positional k-mer matching rather than the abundance of individual UMIs. Therefore, variations in the duplication distribution mainly affect the sizes of the resulting clusters rather than the assignment of reads to clusters. In addition, PCR amplification increases the number of identical reads originating from the same fragment but does not alter their underlying sequence, meaning that duplicated reads should still be grouped into the same cluster regardless of their abundance.

Clustering quality assessment

Comparing the predicted clusters to the model clusters generated from the simulated dataset yielded the following results:

Table 3. Clustering evaluation results for the simulated dataset under three different configurations. For runs including the secondary cycle, a k-mer length of 30 and a distance threshold of 5% were used. The primary cycle consists of anchor-based clustering with UMI subclustering, while the secondary cycle performs the finer clustering.

Metric	Primary only	Secondary only	Both
Pred. Clusters	13,290,236	4,505,486	6,312,607
ARI	0.5374	0.3922	0.9945
Precision	0.9999	0.2467	0.9999
Recall	0.3674	0.9562	0.9891
F1	0.5374	0.3922	0.9945
Homog.	0.9999	0.9601	0.9999
Compl.	0.9540	0.9970	0.9955

The combined clustering approach achieves the best performance across (almost) all metrics. Primary clustering alone shows extremely high precision but low recall, indicating over-fragmentation of clusters, whereas secondary clustering alone produces high recall but substantially lower precision due to over-clustering.

The lower recall observed in the primary-only run can be attributed to mismatches in the last 50 bp of the reads. Given the sequencing error rate of 1% in the simulated dataset, the probability of observing at least one error within a 50 bp fragment is approximately 39%. Consequently, many reads originating from the same fragment may contain sequencing errors in this anchor region. As a result, reads that should belong to the same cluster may be separated into multiple smaller clusters, often differing by only a single nucleotide substitution.

In contrast, the lower precision observed in the secondary-only run can largely be explained by the presence of nearly identical isoforms. In highly expressed genomic regions, multiple transcripts can originate from overlapping or very similar sequences. When clustering relies primarily on sequence similarity without the additional discrimination provided by UMI

information, reads originating from different but highly similar transcripts may be incorrectly merged into the same cluster.

Overall, combining both clustering stages provides the best trade-off between dataset reduction and clustering accuracy, allowing the dataset size to be substantially reduced while preserving the underlying biological signal.

Clustering algorithm

Read hashing

The main computational bottleneck in sequence clustering was the number of comparisons required to assign each incoming read to an existing cluster. A naive strategy would compute the distance between the query read and all previously processed reads. However, this results in quadratic complexity in the number of reads, which becomes infeasible for datasets containing millions of sequences.

A natural improvement is to represent each cluster by a single seed sequence summarizing the reads it contains, reducing the number of comparisons to the number of clusters rather than the number of reads. In practice, however, this approach still proved to be too slow due to the large number of singleton reads - reads that cannot be clustered with any other read - which are common in RNA-seq datasets. As a result, the number of clusters can approach the number of reads.

To address this issue, we introduced a hashing-based candidate retrieval strategy. For each read, a hash value is generated that allows the algorithm to query a subset of clusters that are likely to match the read. The initial implementation used traditional k-mers to construct hash buckets, where each bucket contained clusters sharing a specific k-mer. However, this approach produced too many candidate clusters. Traditional k-mer sets are highly sensitive to mismatches: a single nucleotide substitution can destroy up to k k-mers, while a shift in the sequence only affects two k-mers. Moreover, traditional k-mer sets discard positional information, which further increases the number of potential candidates.

Minimizers were also evaluated as an alternative hashing strategy in order to incorporate partial positional information while reducing the number of stored seeds. Although minimizers slightly improved the specificity of candidate retrieval, they were more expensive to compute and did not sufficiently reduce the number of candidate clusters in practice.

The best performance was achieved using positional k-mers. Instead of generating all $l - k + 1$ k-mers, the read is divided into equally sized segments and a k-mer is extracted from each segment together with its positional index. This positional information significantly increases the specificity of candidate retrieval. In this representation, a single mismatch affects only one k-mer, whereas a positional shift may disrupt multiple k-mers. Consequently, positional k-mers provide a more suitable hashing strategy for read deduplication and substantially reduce the number of candidate clusters that must be examined.

In addition to improving candidate specificity, positional k-mers also increase the memory efficiency of the algorithm. A traditional k-mer hashing scheme requires computing and storing all $l - k + 1$ k-mers for each read. Minimizers reduce this number by selecting representative k-mers within sliding windows. However, this approach introduces an implicit accuracy trade-off, as the window size determines how much positional information is preserved. Positional k-mers avoid this trade-off by storing only a fixed number of position-specific k-mers while retaining explicit positional information.

Parameter tuning

Table 5 summarizes the clustering performance across different k-mer sizes and similarity thresholds. Overall, increasing the threshold consistently improves clustering quality, as reflected by higher Adjusted Rand Index (ARI) and pairwise recall values. In contrast, precision and homogeneity remain close to 1 for all parameter settings, indicating that false positive cluster assignments are rare and that most clusters remain highly pure.

For all tested k-mer sizes, a threshold of 0.05 yields near-optimal performance. At this threshold, ARI values exceed 0.99 for $k \leq 30$, and pairwise recall approaches or exceeds 0.99, indicating that the majority of true cluster relationships are correctly recovered.

The choice of k-mer size introduces a trade-off between clustering accuracy and computational cost. Smaller k values (e.g., $k = 15$ and $k = 10$) achieve the highest ARI and recall, reaching almost perfect clustering performance. However, this comes at the expense of substantially increased runtime and memory consumption. For example, $k = 10$ requires more than 60 GB of peak memory and significantly longer runtime compared to larger k values.

In contrast, larger k values (e.g., $k = 30$) are more computationally efficient but exhibit slightly reduced recall at lower thresholds. Among all configurations, $k = 25$ with a threshold of 0.05 provides the best overall trade-off between clustering quality and efficiency. This setting achieves an ARI of 0.9973 and a pairwise recall of 0.9947, while maintaining relatively low runtime (4.34 minutes) and memory usage (31.20 GB).

Based on these results, we select $k = 25$ and a threshold of 0.05 as the default parameter configuration for subsequent experiments, as it offers near-optimal clustering performance with substantially reduced computational cost compared to smaller k values.

The optimal parameter configuration is expected to vary across datasets, as it depends on factors such as sequencing error rate, read length, and the extent of PCR duplication. In particular, higher error rates or shorter reads may favor smaller k-mer sizes and more permissive thresholds, whereas lower error rates and longer reads allow for more stringent parameter settings.

Benchmarks

To benchmark the program, it was executed on the sample datasets H1 and H5 across 10 independent runs. For each sample, the mean run time and peak memory usage were computed for both R1 and R3 replicates.

According to Figure 11, the dual clustering step, in which subclusters are constructed using both anchor sequences and UMIs, accounts for the largest portion of the total run time. Compared to the other components of the program, this step consistently requires the most processing time across all runs. The clustering procedure involves mutating up to two nucleotides within the UMI sequence to identify the most

suitable cluster match, which results in a large number of comparisons and lookup operations. Although this step employs a rolling hash implementation to efficiently generate mutations and avoid repeated string construction, it still contributes the majority of the computational cost due to the number of comparisons and search operations performed.

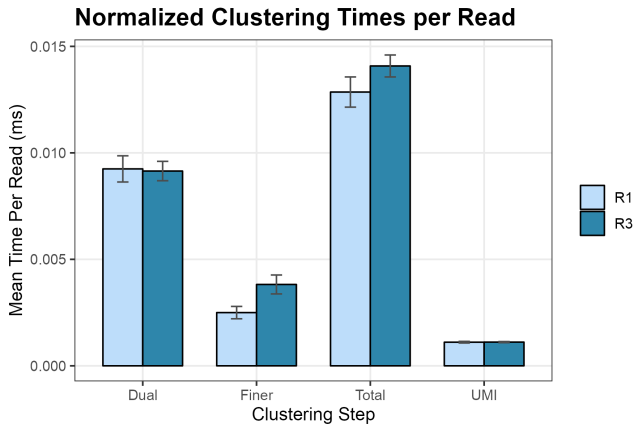

Figure 11 Bar plot showing the mean normalized run time of the program across 10 runs. Run times were normalized by the number of processed reads, resulting in a time-per-read measurement. Run time is reported in milliseconds (ms). Error bars represent the standard deviation across runs. UMI stands for reading and grouping UMIs, Dual stands for anchor-based clustering with UMI-subclustering and finer stands for finer clustering.

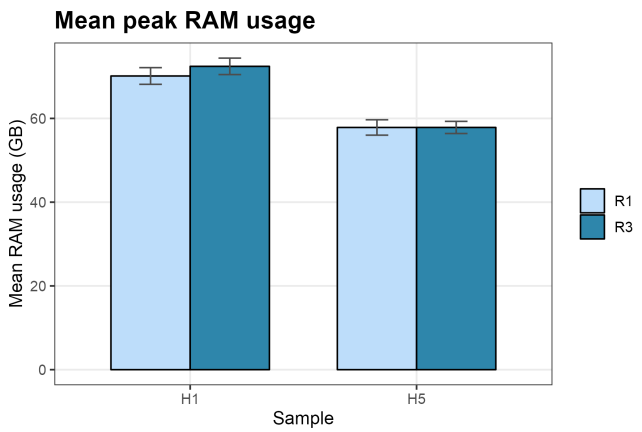

Figure 12 Bar plot showing the mean peak RAM usage of the program across 10 runs for samples H1 and H5. RAM usage is reported in gigabytes (GB). Error bars represent the standard deviation across runs.

The peak RAM usage of the program is relatively high, reaching approximately 70 GB for sample H1 (45.23 million reads) and around 60 GB for sample H5 (27.61 million reads), as shown in Figure 12. The difference between the two samples roughly follows the difference in input size, with the larger dataset requiring more memory. A major contributor to the overall memory consumption is the use of native Java String objects to hash and store UMI sequences, along with several arrays used during consensus sequence construction. Additionally, the hierarchical clustering structure contributes substantially to memory usage. The program creates a large number of cluster and subcluster objects, and in some cases individual clusters contain hundreds of subclusters. Additionally the post processing also creates a new positional k-mer set data structure and k-mers from the consensus sequences to cluster the subclusters together. As a result, the accumulation of millions of cluster-related objects leads to the high overall memory footprint.

Conclusion

In this work, we presented and evaluated a k-mer based clustering approach for deduplicating RNA-seq reads under realistic sequencing conditions. Both simulated and real RNA-seq datasets demonstrated effective deduplication, with the simulated data achieving near-perfect clustering performance.

A limitation of the consensus-based approach is that distinct fragments with identical anchor sequences and similar UMIs may be erroneously merged. However, such collisions are expected to be rare in typical RNA-seq data due to sufficient sequence diversity, and their overall impact on downstream analyses is likely limited [?].

Despite the strong performance observed on simulated data, results on real RNA-seq datasets indicate that clustering remains more challenging in practice. This can be attributed to factors such as sequencing errors and PCR amplification biases. Consequently, while the proposed method is robust, its performance is influenced by the underlying data characteristics, highlighting the importance of careful parameter tuning and dataset-specific considerations.

Acknowledgments

The authors thank Prof. Dr. Ralf Zimmer, Sam Klein, Armin Hadziahmetovic, Elena Kraus and Ida Reichegger for their valuable suggestions and discussions.

References

1. K. R. Kukurba and S. B. Montgomery. RNA Sequencing and Analysis. *Cold Spring Harbor Protocols*, 2015(11): pdb.top084970, Nov. 2015. ISSN 1940-3402, 1559-6095. doi: 10.1101/pdb.top084970. URL `http://www.cshprotocols.org/lookup/doi/10.1101/pdb.top084970`.
2. A. E. Minoche, J. C. Dohm, and H. Himmelbauer. Evaluation of genomic high-throughput sequencing data generated on illumina hiseq and genome analyzer systems. *Bioinformatics*, 27(17):2384–2389, 2011.
3. S. Parekh, C. Ziegenhain, B. Vieth, W. Enard, and I. Hellmann. The impact of amplification on differential expression analyses by RNA-seq. 6(1):25533. ISSN 2045-2322. doi: 10.1038/srep25533. URL `https://doi.org/10.1038/srep25533`.
4. T. Smith, A. Heger, and I. Sudbery. Umi-tools: modeling sequencing errors in unique molecular identifiers to improve quantification accuracy. *Genome Research*, 27(3):491–499,

2017. doi: 10.1101/gr.209601.116. URL https://doi.org/10.1101/gr.209601.116.

5. Z. Wang, M. Gerstein, and M. Snyder. RNA-Seq: a revolutionary tool for transcriptomics. *Nature Reviews Genetics*, 10(1):57–63, Jan. 2009. ISSN 1471-0056, 1471-0064. doi: 10.1038/nrg2484. URL https://www.nature.com/articles/nrg2484.

Supplement

Table 4. Approximate number clusters after deduplication and correction through clustering with UMIs and anchors.

Sample	Before clustering	Anchor clusters R1	Anchor clusters R3	Subclusters R1	Subclusters R3	Finer clusters R1	Finer clusters R3
H1	45.23 M	16.24 M	17.15 M	17.33 M	18.20 M	16.99 M	17.70 M
H2	41.20 M	17.02 M	17.77 M	19.42 M	20.07 M	19.26 M	19.85 M
H3	69.03 M	20.33 M	21.70 M	22.13 M	23.43 M	21.49 M	22.57 M
H4	77.66 M	24.14 M	25.75 M	26.07 M	27.62 M	25.49 M	26.77 M
H5	27.61 M	13.52 M	13.87 M	14.19 M	14.52 M	13.97 M	14.24 M
H6	73.38 M	24.49 M	25.99 M	29.11 M	30.43 M	28.76 M	29.93 M
H7	79.67 M	25.00 M	26.64 M	27.06 M	28.63 M	26.44 M	27.75 M
H8	84.07 M	28.89 M	30.12 M	33.70 M	34.78 M	33.14 M	34.11 M
H9	74.43 M	26.56 M	27.97 M	28.92 M	30.26 M	28.42 M	29.56 M

Table 5. Clustering performance for different k-mer sizes and thresholds.

k-mer	thr	ARI	Prec	Recall	F1	Homo	Compl	V-meas	Time (min)	Mem (GB)
30	0.01	0.6654	0.9999	0.4986	0.6654	1.0000	0.9633	0.9813	4.61	37.66
30	0.02	0.8191	0.9999	0.6937	0.8191	1.0000	0.9770	0.9884	4.59	34.66
30	0.03	0.9640	0.9999	0.9305	0.9640	1.0000	0.9943	0.9971	4.43	33.37
30	0.05	0.9945	0.9999	0.9891	0.9945	1.0000	0.9991	0.9995	4.46	32.11
25	0.01	0.6654	0.9999	0.4986	0.6654	1.0000	0.9633	0.9813	5.05	39.81
25	0.02	0.8191	0.9999	0.6937	0.8191	1.0000	0.9770	0.9884	5.13	37.36
25	0.03	0.9640	0.9999	0.9305	0.9640	1.0000	0.9943	0.9971	4.64	32.82
25	0.05	0.9973	0.9999	0.9947	0.9973	1.0000	0.9995	0.9998	4.34	31.20
15	0.01	0.6654	0.9999	0.4986	0.6654	1.0000	0.9633	0.9813	5.95	50.32
15	0.02	0.8191	0.9999	0.6937	0.8191	1.0000	0.9770	0.9884	5.93	49.78
15	0.03	0.9640	0.9999	0.9305	0.9640	1.0000	0.9943	0.9971	5.19	41.73
15	0.05	0.9978	0.9999	0.9956	0.9978	1.0000	0.9996	0.9998	5.74	42.64
10	0.01	0.6654	0.9999	0.4986	0.6654	1.0000	0.9633	0.9813	7.15	65.75
10	0.02	0.8191	0.9999	0.6937	0.8191	1.0000	0.9770	0.9884	8.53	63.42
10	0.03	0.9640	0.9999	0.9305	0.9640	1.0000	0.9943	0.9971	6.41	60.22
10	0.05	0.9978	0.9999	0.9956	0.9978	1.0000	0.9996	0.9998	6.04	55.42